JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Enum StandardOpenOption

java.lang.Object
    java.lang.Enum<StandardOpenOption>
        java.nio.file.StandardOpenOption

All Implemented Interfaces:

Serializable, Comparable<StandardOpenOption>, OpenOption

```
public enum StandardOpenOption
extends Enum<StandardOpenOption>
implements OpenOption
```

Defines the standard open options.

Since:

1.7

Enum Constant Summary

Enum Constants

Enum Constant	Description
APPEND	If the file is opened for **WRITE** access then bytes will be written to the end of the file rather than the beginning.
CREATE	Create a new file if it does not exist.
CREATE_NEW	Create a new file, failing if the file already exists.
DELETE_ON_CLOSE	Delete on close.
DSYNC	

	Requires that every update to the file's content be written synchronously to the underlying storage device.
READ	Open for read access.
SPARSE	Sparse file.
SYNC	Requires that every update to the file's content or metadata be written synchronously to the underlying storage device.
TRUNCATE_EXISTING	If the file already exists and it is opened for **WRITE** access, then its length is truncated to 0.
WRITE	Open for write access.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static **StandardOpenOption**	**valueOf**(**String** name)	Returns the enum constant of this type with the specified name.
static **StandardOpenOption**[]	**values**()	Returns an array containing the constants of this enum type, in the order they are declared.

Methods inherited from class java.lang.Enum

clone, compareTo, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf

Methods inherited from class java.lang.Object

getClass, notify, notifyAll, wait, wait, wait

Enum Constant Detail

READ

```
public static final StandardOpenOption READ
```

Open for read access.

WRITE

```
public static final StandardOpenOption WRITE
```

Open for write access.

APPEND

```
public static final StandardOpenOption APPEND
```

If the file is opened for `WRITE` access then bytes will be written to the end of the file rather than the beginning.

If the file is opened for write access by other programs, then it is file system specific if writing to the end of the file is atomic.

TRUNCATE_EXISTING

```
public static final StandardOpenOption TRUNCATE_EXISTING
```

If the file already exists and it is opened for `WRITE` access, then its length is truncated to 0. This option is ignored if the file is opened only for `READ` access.

CREATE

```
public static final StandardOpenOption CREATE
```

Create a new file if it does not exist. This option is ignored if the `CREATE_NEW` option is also set. The check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other file system operations.

CREATE_NEW

```
public static final StandardOpenOption CREATE_NEW
```

Create a new file, failing if the file already exists. The check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other file system operations.

DELETE_ON_CLOSE

```
public static final StandardOpenOption DELETE_ON_CLOSE
```

Delete on close. When this option is present then the implementation makes a *best effort* attempt to delete the file when closed by the appropriate `close` method. If the `close` method is not invoked then a *best effort* attempt is made to delete the file when the Java virtual machine terminates (either normally, as defined by the Java Language Specification, or where possible, abnormally). This option is primarily intended for use with *work files* that are used solely by a single instance of the Java virtual machine. This option is not recommended for use when opening files that are open concurrently by other entities. Many of the details as to when and how the file is deleted are implementation specific and therefore not specified. In particular, an implementation may be unable to guarantee that it deletes the expected file when replaced by an attacker while the file is open. Consequently, security sensitive applications should take care when using this option.

For security reasons, this option may imply the `LinkOption.NOFOLLOW_LINKS` option. In other words, if the option is present when opening an existing file that is a symbolic link then it may fail (by throwing `IOException`).

SPARSE

```
public static final StandardOpenOption SPARSE
```

Sparse file. When used with the `CREATE_NEW` option then this option provides a *hint* that the new file will be sparse. The option is ignored when the file system does not support the creation of sparse files.

SYNC

```
public static final StandardOpenOption SYNC
```

Requires that every update to the file's content or metadata be written synchronously to the underlying storage device.

See Also:

Synchronized I/O file integrity

DSYNC

```
public static final StandardOpenOption DSYNC
```

Requires that every update to the file's content be written synchronously to the underlying storage device.

See Also:

Synchronized I/O file integrity

Method Detail

values

```
public static StandardOpenOption[] values()
```

Returns an array containing the constants of this enum type, in the order they are declared. This method may be used to iterate over the constants as follows:

```
for (StandardOpenOption c : StandardOpenOption.values())
    System.out.println(c);
```

Returns:

an array containing the constants of this enum type, in the order they are declared

valueOf

```
public static StandardOpenOption valueOf(String name)
```

Returns the enum constant of this type with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this type. (Extraneous whitespace characters are not permitted.)

Parameters:

name - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

IllegalArgumentException - if this enum type has no constant with the specified name

```
NullPointerException - if the argument is null
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.